

Technical Report: Hybrid Quantum Circuit Simulation

1. Implementation Approach and State Representation

The simulation engine models quantum states using linear algebra primitives, establishing a foundation that seamlessly bridges wave mechanics and classical computational modeling. A single qubit is represented as a 2D column vector storing the complex probability amplitudes α and β , strictly satisfying the normalization condition:

$$|\alpha|^2 + |\beta|^2 = 1$$

Superposition and entanglement are modeled by projecting the baseline vectors $|0\rangle$ and $|1\rangle$ through transformation matrices.

- **Pauli-X (NOT) Gate:** Executed as a matrix multiplication that flips the state vector amplitudes.
- **Hadamard (H) Gate:** Forces a localized probability distribution into an equal superposition, representing fundamental quantum uncertainty.
- **CNOT Gate:** Modeled as a 4x4 transformation matrix over a 4D state vector (created via Kronecker product).

2. Circuit Explanation and KPI Discussion

To evaluate multi-qubit execution, the simulator executes a standard Bell State workflow.

Execution Workflow:

1. A Hadamard gate is applied to qubit 0, transforming it into a superposition state.
2. A CNOT gate acts on the unified system, utilizing qubit 0 as the control and qubit 1 as the target.
3. The resulting entangled state yields a probability distribution of exactly 0.50 for $|00\rangle$ and 0.50 for $|11\rangle$.

KPI Evaluation:

System metrics were captured using standard process timers and memory profiling.

- **Execution Time:** The application of 2x2 and 4x4 matrices executes in microseconds, well within acceptable latency bounds for standard CPU processing.

- **State Vector Scaling:** A single-qubit system requires a 2-element vector, while a two-qubit system requires a 4-element vector. The scaling behaves exponentially (2^n , where n represents the number of simulated qubits).
- **Memory Footprint:** While classical memory easily stores the 2-qubit footprint, the geometric scaling dictates that simulating systems around 50 qubits will exhaust all available RAM due to the rapid explosion of probability amplitudes.

Note on Scaling: The 2^n growth factor is the primary bottleneck defining the limits of classical simulation of quantum systems.

3. Hardware Analysis and FPGA Feasibility

Translating this software model into dedicated silicon requires optimizing the logic for a Field Programmable Gate Array (FPGA) or custom electronic instrumentation. The current CPU-bound simulator is highly inefficient for complex matrix multiplication.

- **Fixed-Point vs. Floating-Point Arithmetic:** The Python simulator natively relies on floating-point operations. Synthesizing full floating-point logic pipelines on an FPGA consumes massive swathes of finite lookup tables and logic gates. Transitioning the arithmetic core to fixed-point numeric representation drastically compresses the logic footprint and accelerates gate throughput, though it introduces slight precision quantization errors in the terminal probability calculations.
- **Matrix Parallelism:** Quantum gate operations map cleanly to parallelized matrix dot products. Standard CPU architecture evaluates these sequentially. A custom VLSI architecture can execute these row-column operations concurrently within a single clock cycle, maximizing hardware throughput.
- **Memory Architectures:** The exponential growth of the state vector creates severe memory bandwidth bottlenecks between the logic fabric and external DDR. Implementing a sparse storage technique—allocating on-chip block RAM only for non-zero amplitudes—directly mitigates bandwidth saturation and enables the physical simulator to support more qubits.

4. Failure Analysis and Limitations

The primary failure mode of this software simulator is its inability to scale polynomially. As qubit counts increase, the exponential memory requirement (2^n) directly throttles runtime and crashes standard compute nodes. Furthermore, matrix operations on classical CPUs suffer from cache-miss penalties as the state vectors rapidly exceed standard L1/L2 caching boundaries.

For future iterations targeting hardware acceleration, addressing fixed-point rounding errors is critical. While fixed-point math enables practical FPGA porting, the data width must be

rigorously calibrated against the necessary fidelity of the quantum simulation to prevent compounding errors from collapsing fragile probability measurements over deep circuits.